

You might think that the simplest way to write software for an SoC would be to fire up Android Studio/XCode and create an App that displays the famous “hello digit” program (did you notice what we did there? :P). But that’s not how software for SoCs are written!

Apps written for a mobile platform would not qualify as a software component for a SoC. Apps (as the name suggests,) are application software. They need a complete operating system and execution environment to run. E.g. an android app would need a execution environment for the application code (Dalvik or ART) to run. If you were to take the binary code from that app and attempt to run it on the SoC processor, you would not get the desired result. It would appear as crash! For a piece of software to be called a software component for the SoC, it should be closely tied to the SoC. Let’s dig a little deeper.

Layers in Software – a primer

With all the buzzwords thrown around, it is easy to be trapped into thinking that an SoC is the mobile phone and SoCs were designed only to be used in mobiles. We have already expressed that to be untrue – that mobiles need more hardware than just the SoC and that there are SoCs implemented in devices other than mobiles. However, smartphones are the most popular implementation; thus, we will start with them.

An SoC is a system and by system, we mean a computing system. Software written for any system that can perform generic tasks are logically layered. The part of a software which interacts directly with hardware is said to be lower-layer software. It normally consists of the kernel and drivers. These parts know how to talk to the hardware (which is why they are called lower-level code). All other software depend on these low-level software components. For example: a Bluetooth driver knows the protocol its hardware unit follows and thus can ask it to send a pairing request signal. The UI elements which allow you to pair your wireless headsets would not be able to do anything if the driver did not know how to tell the Bluetooth controller to initiate pairing with a new device.

On the same level reside other critical components of the OS which includes process schedulers, memory managers, interrupt handlers, network stacks, display managers etc. Depending on hardware components built into the SoC and the implementation and usage, these parts may or may not be present. The kernel (another name for core) of an OS contains code that can interact with the microprocessor. Drivers for devices attached to the SoC also execute with low-level privilege.